Docker Compose & Container Configuration

Relevant source files

- `Backend/STUDER-MEDIA/Dockerfile`
- `Backend/STUDER-MEDIA/main.py`
- `Backend/STUDER-MEDIA/placeholder.txt`
- `Backend/STUDER/Dockerfile`
- `Backend/STUDER/pom.xml`
- `Docker/docker-compose.yml`
- `Docker/nginx.conf`

This page details the containerized infrastructure of the STUDER project. The system is orchestrated using Docker Compose, employing a microservices-adjacent architecture that separates the web server, application logic, media processing, and storage layers into isolated containers.

Docker Compose Services

The **`docker-compose.yml`** file defines six primary services and two persistent volumes to ensure data consistency across restarts `Docker/docker-compose.yml 1-94`

Service Map

Service	Container Name	Image / Build Source	Network	Role
`nginx`	**`studer_nginx`**	**`nginx:alpine`**	**`frontend_net`**, **`backend_net`**	Reverse proxy and entry point `Docker/docker-compose.yml 59-72`
`app-be`	**`studer_app_be`**	**`../Backend/STUDER/`**	**`backend_net`**	Spring Boot Java backend `Docker/docker-compose.yml 17-35`
`frontend`	**`studer_frontend`**	**`../Frontend/STUDER/`**	**`frontend_net`**	Angular SPA `Docker/docker-compose.yml 51-58`

Service	Container Name	Image / Build Source	Network	Role
`studer-media`	`studer_media`	`../Backend/STUDER-MEDIA/`	`backend_net`	Python FastAPI media processor `Docker/docker-compose.yml 36-50`
`minio`	`studer_minio`	`minio/minio`	`backend_net`	S3-compatible object storage `Docker/docker-compose.yml 73-86`
`db`	`studer_db`	`postgres:16`	`backend_net`	Relational database `Docker/docker-compose.yml 3-16`

Network Topology and Isolation

STUDER utilizes two bridge networks to enforce security boundaries:

- **frontend_net**: Connects the `nginx` proxy to the `frontend` container `Docker/docker-compose.yml 91-92`
- **backend_net**: Connects the `nginx` proxy to the `app-be` service, and allows internal communication between `app-be`, `studer-media`, `minio`, and `db` `Docker/docker-compose.yml 93-94`

Infrastructure Network Flow

Sources: `Docker/docker-compose.yml 1-94` `Docker/nginx.conf 28-51`

Backend (app-be) Configuration

The Spring Boot backend uses a multi-stage Dockerfile to optimize the final image size by separating the build environment from the runtime environment.

Multi-Stage Dockerfile Implementation

1. **Build Stage**: Uses `maven:3.9.6-eclipse-temurin-17-alpine` to compile the application and package it into a JAR file, skipping tests to accelerate the build

`Backend/STUDER/Dockerfile 1-6`

2. **Runtime Stage**: Uses `eclipse-temurin:17-jre-alpine` as a lightweight base. It copies only the generated `STUDER-0.0.1-SNAPSHOT.jar` from the build stage `Backend/STUDER/Dockerfile 8-13`

Environment Variable Wiring

The `app-be` service translates Docker environment variables into Spring properties to configure the production profile `Docker/docker-compose.yml 22-30`

Variable	Usage	Source
`SPRING_PROFILES_ACTIVE`	Sets profile to `prod`	`Docker/docker-compose.yml 23`
`POSTGRES_HOST`	Set to `db` (service name)	`Docker/docker-compose.yml 27`
`JWT_SECRET`	Secret key for token signing	`Docker/docker-compose.yml 29`
`APP_MEDIA_SERVICE_URL`	`http://studer-media:8000`	`Docker/docker-compose.yml 30`

Sources: `Backend/STUDER/Dockerfile 1-14` `Docker/docker-compose.yml 17-35`

Media & Storage Configuration

The media subsystem consists of the `studer-media` FastAPI service and the `minio` object store.

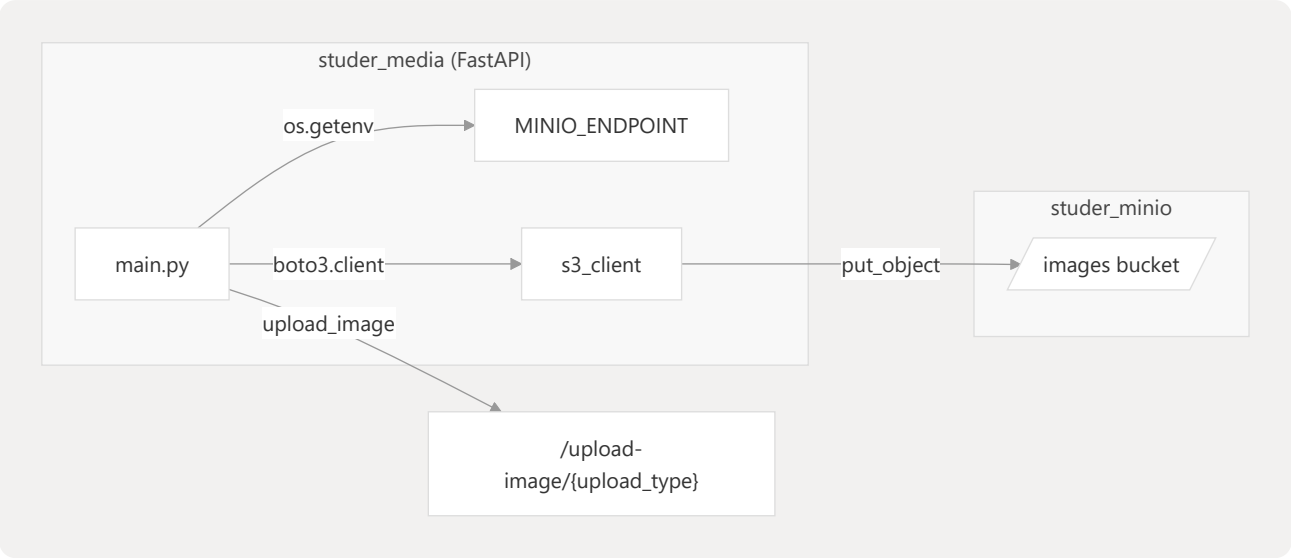
Media Service (studer-media)

The service is built from the Python source `Backend/STUDER-MEDIA/Dockerfile 1-11` On startup, it executes `create_bucket_if_not_exists()` to ensure the `images` bucket is available and configured with a public-read policy via `set_public_read_policy` `Backend/STUDER-MEDIA/main.py 60-95`

MinIO Persistence

MinIO is configured to store data in a persistent volume named `minio_data` `Docker/docker-compose.yml 79-80` It provides both the API port (`9000`) and the web console port (`9001`) `Docker/docker-compose.yml 76-78`

Media Storage Code Entity Map



Sources: `Backend/STUDER-MEDIA/main.py 18-40` `Backend/STUDER-MEDIA/main.py 97-102` `Docker/docker-compose.yml 73-86`

Volume Management & Persistence

The configuration defines two named volumes to ensure that user data and uploaded media survive container restarts or removals.

- **db_data**: Mounted at `/var/lib/postgresql/data` within the `db` container to persist the PostgreSQL database files `Docker/docker-compose.yml 14-88`
- **minio_data**: Mounted at `/data` within the `minio` container to persist all objects (images) stored in the S3 buckets `Docker/docker-compose.yml 80-89`

Both services are configured with `restart: always` to ensure high availability within the Docker daemon `Docker/docker-compose.yml 6-69`

Sources: `Docker/docker-compose.yml 1-94`